

ArmorIQ Hackathon

Personal Assistant Agentic Systems

The Problem

AI personal assistants are rapidly moving beyond conversational help. They now plan trips, manage subscriptions, coordinate households, submit forms, and spend real money on behalf of users. While this creates convenience and efficiency, it also introduces a new class of risk.

Most agentic systems today optimize for helpfulness first and safety later. When given broad permissions, agents can easily exceed what users expect. A request such as “handle this for me” can lead to unintended purchases, canceled services, or irreversible actions. Once agents operate autonomously, mistakes are no longer hypothetical. They directly impact trust, finances, and user control.

This hackathon explores a core question:

How do we build AI personal assistant agents that are genuinely useful while ensuring users remain in control of what those agents can do?

Your Challenge

Teams will build a personal assistant style agentic system that helps users with everyday tasks while enforcing strong boundaries by design.

You may choose any scenario that matters to you. The focus is not on building the most features, but on demonstrating clear guarantees around how agents act in the real world.

What matters is not the feature set, but the guarantees your system provides.

Your system should be structured around:

- Agents that perform reasoning, planning, and coordination
- MCP servers that expose real world capabilities such as payments, bookings, submissions, or updates
- Clear boundaries where all real world actions occur only through MCP servers that enforce user defined rules

Users do not approve every action interactively. Instead, users define rules that specify what agents are allowed to do. The system is responsible for enforcing those rules correctly and consistently.

Recommended Tooling and Onboarding

Teams are free to use any framework or platform of their choice, provided the implementation is done in **Python or TypeScript**. For MCP <> Agent scenarios, LangChain agents are recommended.

OpenClaw runs locally on user devices and integrates with messaging platforms. Originally released in November 2025 under the name Clawdbot, it was renamed Moltbot before the final name. OpenClaw operates as an autonomous agent that can execute tasks through messaging platforms such as WhatsApp, Telegram, and Signal, enabling automated workflows across multiple services. Moltbots have demonstrated the capabilities of autonomous AI agents and the practical impact they can have in real world settings. Building on this work, the teams may choose to reuse or build upon for this hackathon.

The OpenClaw framework also has uncovered serious issues including around security, governance etc. When combined with ArmorIQ, these interfaces can be used to enforce clear guarantees around what agents are allowed to do. Teams can optionally use the ArmorIQ extension in aiq-open claw to provide intent and policy guarantees, ensuring that autonomous behavior remains aligned with user defined rules.

Use of the OpenClaw framework is entirely optional. Teams are encouraged to adopt, adapt, or ignore it based on what best supports their system design.

Example Scenarios

The scenarios below are illustrative and non binding. Teams are encouraged to adapt these or propose their own, as long as the same guarantees around constrained actions and bounded delegation are preserved.

1. **Travel Planning and Booking Assistant**, An agent plans a multi city trip by researching routes, comparing prices, and proposing itineraries. Booking flights or hotels is allowed only if rules permit bookings within a defined budget, destination scope, and time window. If the plan exceeds limits or changes unexpectedly, booking actions must be blocked.
2. **Household Subscription and Expense Manager**. An agent monitors recurring subscriptions and household expenses, identifies unused services, and suggests cancellations or downgrades. Any changes to billing accounts must pass through MCP servers enforcing spend caps and account specific rules.
3. **Delegated Grocery and Household Logistics Assistant**. A user temporarily allows another person's agent to place grocery orders on their behalf. The delegated agent can act only within approved stores, categories, time windows, and price limits. It cannot expand scope or perform unrelated actions.
4. **Personal Finance and Bill Pay Assistant**. An agent tracks upcoming bills, suggests payment schedules, and flags unusual charges. Payments are allowed only within

transaction limits, merchant allowlists, and monthly spend caps. Attempts to exceed limits must be blocked.

5. **Email Management and Action Assistant.** An agent helps manage email by summarizing threads, identifying tasks, and drafting responses. It can send emails, archive threads, or schedule follow ups only within defined recipient lists, domains, and action types. It cannot send messages on the user's behalf to new contacts or take irreversible actions without explicit allowance.
6. **Social Media Interaction Assistant.** An agent assists with managing social media presence by drafting posts, responding to comments, and scheduling content. It can act only on approved platforms, within tone and content guidelines defined by the user. Posting, replying, or deleting content outside these rules must be blocked.

What Your System Must Demonstrate

Your system should clearly show that agents can reason freely while remaining strictly constrained in how they act.

Submissions should demonstrate:

- A clean separation between agent reasoning and real world execution
- All actions flowing through MCP servers that enforce user defined rules
- Clear blocking behavior when actions violate those rules
- Traceability from agent plans to executed actions
- At least one well scoped delegation scenario where another agent acts on a user's behalf without exceeding granted authority.

What You Will Submit

All teams must submit a **3 minute video demo** showing:

- The chosen scenario and motivation
- The agent and MCP server architecture
- A demo of the system in action
- How intent, policy, and delegation are enforced

Top 5 finalist teams will be invited for a **15 minutes live presentation**, including Q&A, where they will go deeper into their design decisions and system behavior.

What We Care About:

Strong projects will demonstrate:

- a compelling use case for personal assistants
- an agentic system built with clear intent and policy guarantees that prevent unauthorized actions
- a thoughtful delegation scenario that is well scoped and well enforced

Awards

- The winning team will receive 50k INR
- The runner up team will get 30k INR
- The finalists get a chance to intern/work full time with the RnD team at ArmorIQ

Resources

- docs.armoriq.ai (every docs)
- platform.armoriq.ai
- <https://docs-openclaw.armoriq.ai/> - AIQ-OPENCLAW-DOCS